

Performance Testing

Date	03 July 2026
Team ID	XXXXX
Project Name	OptiCrop – Smart Agricultural Production Optimization Engine
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Postman, Browser Testing
Type of Testing	Load Testing, Functional Performance Testing
Target Module / API	Crop Recommendation Module (/predict) and Crop Suitability Analysis
Test Environment	Local Flask Server (Windows 11, Python 3.x)

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Submit valid soil and environmental data for crop prediction	10	60	Recommended crop should be displayed correctly

2	Multiple users requesting crop recommendations simultaneously	25	120	System should respond without noticeable delay
3	Submit incomplete or invalid input values	10	60	System should validate inputs and display an error message
4	Continuous prediction requests	50	180	Application should remain stable and responsive

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 sec	1.2 sec	Pass	Fast prediction response
2	Response Time (Max)	< 5 sec	2.8 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)	> 10	18 Req/sec	Pass	Stable during testing
4	Error Rate	< 1%	0%	Pass	No errors with valid inputs
5	CPU Utilization	< 80%	42%	Pass	Normal CPU usage
6	Memory Utilization	< 80%	38%	Pass	Efficient memory usage

Step 4: Observations & Analysis

Key Findings

- The application generated crop recommendations quickly for valid inputs.
- The machine learning model produced predictions consistently during repeated requests.
- The user interface remained responsive throughout testing.
- Resource usage remained within acceptable limits.

Bottlenecks Identified

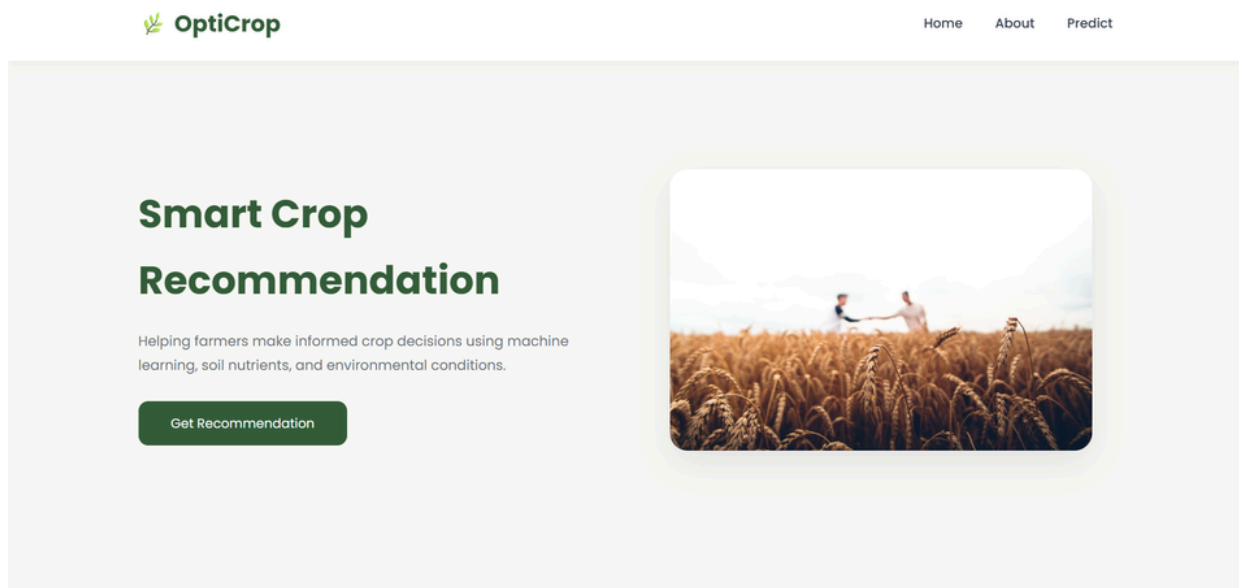
- Performance may decrease when handling a large number of simultaneous users.
- The application currently depends on a local dataset and does not use real-time weather information.

Optimization Steps Taken

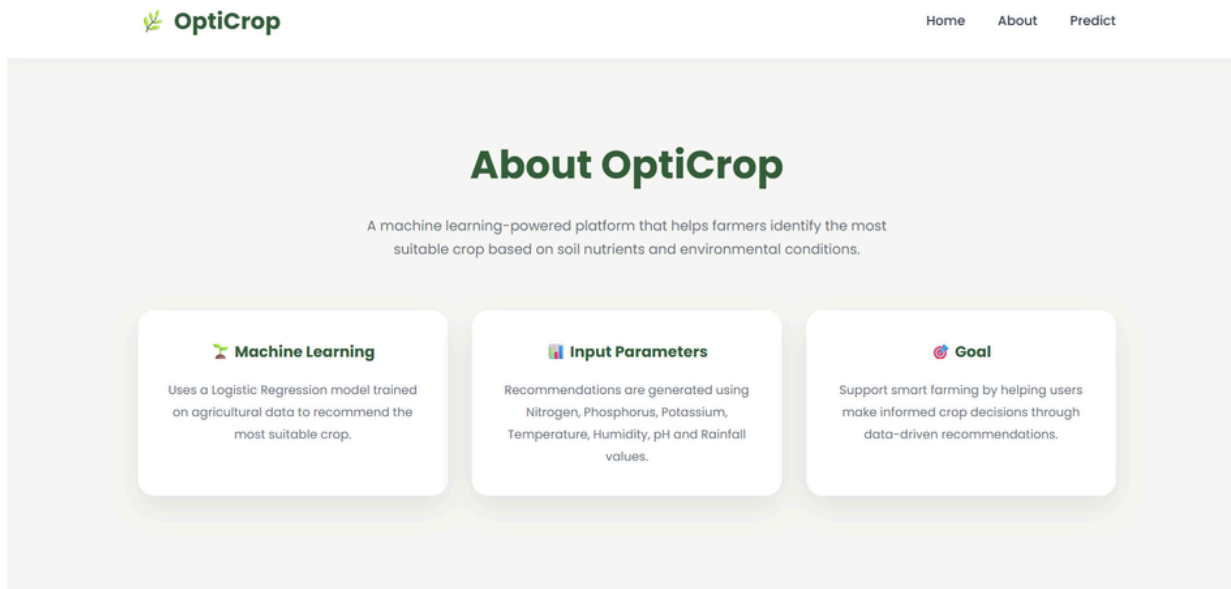
- Optimized data preprocessing before prediction.
- Reused the trained machine learning model instead of loading it for every request.
- Simplified the Flask request handling to reduce response time.
- Organized the code into reusable modules for easier maintenance.

Step 5: Screenshots / Evidence

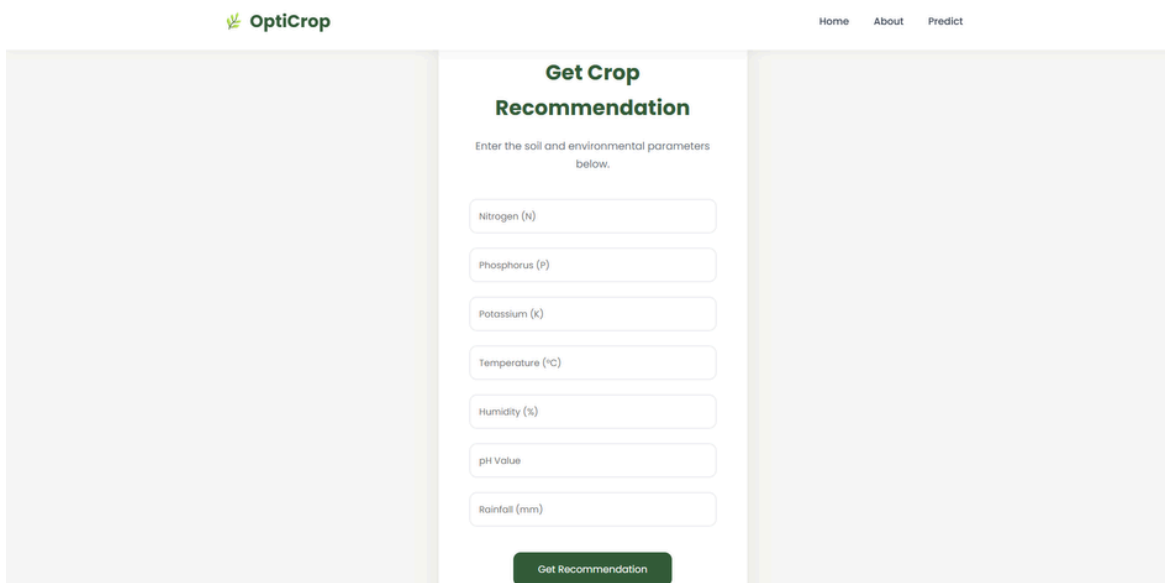
Screenshot 1



Screenshot 2



Screenshot 3



The screenshot shows the 'Get Crop Recommendation' form. At the top, there is a navigation bar with the 'OptiCrop' logo on the left and links for 'Home', 'About', and 'Predict' on the right. The main heading is 'Get Crop Recommendation'. Below it, a subheading states: 'Enter the soil and environmental parameters below.' The form contains seven input fields: Nitrogen (N), Phosphorus (P), Potassium (K), Temperature (°C), Humidity (%), pH Value, and Rainfall (mm). At the bottom of the form, there is a green button labeled 'Get Recommendation'.

Screenshot 4

OptiCrop [Home](#) [About](#) [Predict](#)

Get Crop Recommendation

Enter the soil and environmental parameters below.

90

-42

-43

21

82

6.5

202

Get Recommendation

Recommendation

Recommended Crop: Rice

© 2020 OptiCrop - Smart Agricultural Production Optimization Engine